1 / 5 • Objetivo e organização

O campo de senha por dentro

O que foi feito

O campo de senha recebeu um botão com ícone de olho no lado direito. A senha começa oculta. Ao clicar no botão, o texto aparece; ao clicar novamente, ele volta a ficar oculto. O cadeado que já existia continua no lado esquerdo.

A implementação está em src/components/MainForms/index.tsx, dentro do componente MainForms. Não foi criado um componente separado chamado PasswordField: o comportamento foi acrescentado ao TextField de senha que já fazia parte do formulário.

As três peças principais

1. Memória: o useState do React guarda se a senha está visível. O valor é um booleano: true ou false. Ele representa a visibilidade, e não a senha digitada.

2. Exibição: a propriedade type do TextField recebe "text" quando a senha deve aparecer e "password" quando deve ficar oculta.

3. Interação: um IconButton recebe o clique e inverte o estado. O ícone e o texto de acessibilidade acompanham o mesmo estado.

O caminho de um clique

```
Clique no olho
    -> inverte showPassword
    -> React renderiza novamente
    -> type, ícone e descrição são atualizados
```

O que cada biblioteca faz

React fornece o estado e atualiza a interface. Material UI fornece TextField, InputAdornment e IconButton. O pacote @mui/icons-material fornece Visibility e VisibilityOff. São peças combinadas no mesmo campo, sem precisar procurar ou modificar o input manualmente no DOM.

Este guia acompanha o código atual, incluindo useState<boolean>(false). Os trechos menores explicam partes da implementação; na última página há um exemplo completo e isolado para estudar.

2 / 5 • Estado do React

A lógica: guardar e inverter

Criando a memória do componente

```
import { useState } from "react";

const [showPassword, setShowPassword] =
  useState<boolean>(false);
```

useState cria um estado que o React mantém entre renderizações. O argumento false define o início: a senha fica oculta quando o formulário é montado. O tipo <boolean> informa ao TypeScript que o estado aceita true ou false. Neste caso, o TypeScript também conseguiria inferir esse tipo pelo valor inicial.

O par entre colchetes usa desestruturação de array. showPassword é o valor disponível na renderização atual. setShowPassword é a função usada para pedir sua atualização. Os nomes poderiam ser outros, mas estes indicam claramente sua finalidade.

O que acontece no clique

```
onClick={() => setShowPassword((show) => !show)}
```

A função de onClick é executada quando o botão é acionado. Ela chama setShowPassword e passa outra função: (show) => !show. O React fornece a essa função o estado pendente mais recente. O operador ! faz a negação: !false resulta em true e !true resulta em false.

show é apenas o nome do parâmetro dessa função; não é uma segunda variável de estado. Usar a forma funcional é adequado porque o próximo valor depende do valor anterior. Também evita depender de um valor capturado por uma renderização anterior quando há atualizações enfileiradas.

Simulando duas interações

```
Início:       showPassword = false
1º clique:    !false = true   -> aparece
2º clique:    !true  = false  -> oculta
```

Uma variável comum, como let showPassword = false, não substitui esse estado: alterar uma variável local não pede uma nova renderização e ela seria recriada quando a função do componente rodasse novamente. O setter é a ligação entre a mudança e a atualização da interface.

Essa memória existe enquanto o componente permanece montado. Ela não usa localStorage nem salva a escolha em um servidor. Uma nova montagem começa novamente com false.

Montando o campo e o botão

Alternando o tipo do input

```
type={showPassword ? "text" : "password"}
```

Essa expressão usa o operador ternário: condição ? valorSeVerdadeiro : valorSeFalso. Quando showPassword vale true, o input recebe type="text" e mostra os caracteres. Quando vale false, recebe type="password" e o navegador mascara os caracteres.

O campo continua sendo o mesmo elemento na estrutura do React; muda a propriedade que controla sua apresentação. A implementação não apaga nem substitui o texto digitado ao alternar o estado.

Encaixando elementos nas extremidades

```
slotProps={{
  input: {
    startAdornment: (/* cadeado à esquerda */),
    endAdornment: (/* botão à direita */),
  },
}}
```

slotProps permite configurar partes internas do TextField. A opção input configura o componente de entrada do Material UI. startAdornment acrescenta o cadeado no início; endAdornment acrescenta o botão no final. O InputAdornment organiza esses elementos junto ao campo.

```
<InputAdornment position="end">
  <IconButton edge="end" ...>
    {showPassword ? <VisibilityOff /> : <Visibility />}
  </IconButton>
</InputAdornment>
```

position="end" informa que o adorno fica no final da entrada. IconButton cria um botão próprio para um ícone, com suporte à interação de um botão. edge="end" ajusta sua margem para alinhá-lo à extremidade do campo. O texto "..." no trecho acima apenas abrevia propriedades explicadas na próxima página.

Por que o ícone também muda?

Com a senha oculta, aparece Visibility, o olho aberto, indicando a ação de mostrar. Com a senha visível, aparece VisibilityOff, o olho cortado, indicando a ação de ocultar. Assim, o desenho representa o que o próximo clique fará.

O estilo existente em sx foi mantido, incluindo borderRadius: "12px". O arredondamento cuida da aparência; useState, type e onClick cuidam do comportamento.

Clique, foco e acessibilidade

Dando um nome ao ícone

```
aria-label={
  showPassword ? "Ocultar senha" : "Mostrar senha"
}
aria-pressed={showPassword}
```

Um desenho de olho, sozinho, não oferece um nome útil para quem usa leitor de tela. aria-label fornece esse nome. O texto acompanha a ação disponível: "Mostrar senha" enquanto ela está oculta e "Ocultar senha" enquanto está visível.

aria-pressed expõe o estado pressionado de um botão de alternância. Aqui, true corresponde à visualização ativada. Essa propriedade comunica o estado; ela não executa a mudança. O comportamento continua sendo responsabilidade do onClick.

Uma alternativa de acessibilidade para um toggle é manter um nome fixo, como "Exibir senha", e deixar aria-pressed comunicar a alternância. O código atual usa um nome dinâmico junto desse estado; vale avaliar a clareza dessa combinação com leitores de tela ao evoluir o formulário.

Mantendo a atenção no campo

```
onMouseDown={(event) => event.preventDefault()}
```

Ao pressionar o mouse em um botão, o navegador normalmente pode transferir o foco para ele. preventDefault cancela essa ação padrão do mousedown, ajudando a manter o foco no campo durante o uso com mouse. Isso permite continuar digitando sem precisar selecionar a entrada novamente.

Esse evento não alterna a senha e não é o mesmo evento do clique. O onClick continua responsável pela alternância. Também não é necessário adicionar onMouseUp para reproduzir a lógica atual. O comportamento com toque pode variar e deve ser conferido no navegador usado.

Como conferir na interface

Digite uma senha de exemplo: ela deve começar mascarada. Clique no olho: o mesmo texto deve aparecer. Clique novamente: ele deve voltar a ficar oculto. Repita enquanto digita para conferir que o conteúdo permanece.

Use Tab para chegar ao botão e tente acioná-lo com Espaço ou Enter. Confira a mudança do ícone e, com leitor de tela, a descrição anunciada. Estas são orientações de verificação manual; este documento não afirma que esses testes de navegador foram executados.

5 / 5 • Código reunido e limites da implementação

Exemplo completo para estudar

Este exemplo isola o comportamento em um componente pequeno. No projeto, a mesma lógica está dentro de MainForms. O cadeado e os estilos foram omitidos aqui para facilitar a leitura.

```
import { useState } from "react";
import { TextField, InputAdornment, IconButton }
  from "@mui/material";
import Visibility from "@mui/icons-material/Visibility";
import VisibilityOff from "@mui/icons-material/VisibilityOff";

export default function CampoSenha() {
  const [showPassword, setShowPassword] =
    useState<boolean>(false);

  return (
    <TextField
      label="Sua senha"
      type={showPassword ? "text" : "password"}
      slotProps={{
        input: {
          endAdornment: (
            <InputAdornment position="end">
              <IconButton
                aria-label={showPassword
                  ? "Ocultar senha" : "Mostrar senha"}
                aria-pressed={showPassword}
                onClick={() =>
                  setShowPassword((show) => !show)}
                onMouseDown={(e) => e.preventDefault()}
                edge="end"
              >
                {showPassword
                  ? <VisibilityOff /> : <Visibility />}
              </IconButton>
            </InputAdornment>
          ),
        },
      }}
    />
  );
}
```

O que essa implementação resolve

Ela controla a visibilidade local da senha. Não valida credenciais, não envia dados e não implementa autenticação. O estado guarda somente a escolha de mostrar ou ocultar; o valor digitado não é controlado por um estado React neste trecho.

Para lembrar: o clique muda o estado; o estado escolhe o type; o React atualiza o campo. O mesmo estado também mantém o ícone e a descrição alinhados ao comportamento.